

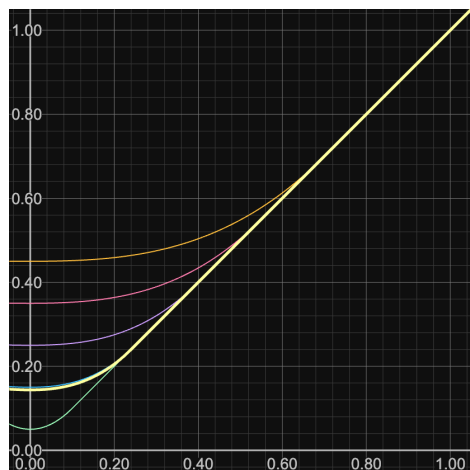
Inigo Quilez :: articles :: useful little functions



Intro

When writing shader or during any procedural creation process (texturing, modeling, shading, animation...) you often find yourself modifying signals in different ways so they behave the way you need. It is common to use `smoothstep()` to threshold some values, or `pow()` to shape a signal, or `clamp()` to clip it, `mod()` to make it repeat, a `mix()` to blend between two signals, `exp()` for attenuation, etc etc. All these functions are often conveniently available by default in most languages. However there are some operations that are also relatively used that don't come by default in any language. The following is a list of some of the functions (of families of functions rather) that I find myself using over and over again. You have the Graphtoy links in each so you can explore them interactively:

Identities

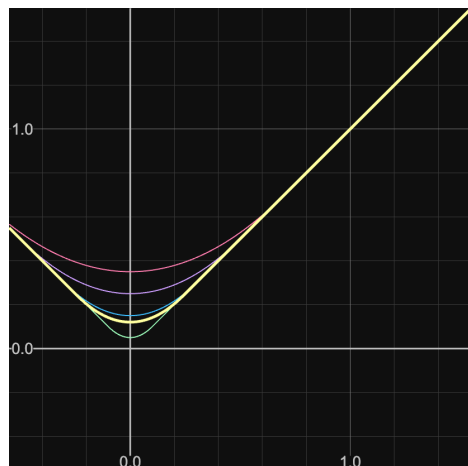


Almost Identity

[graphtoy.com example](#)

Imagine you want to clip a signal **x** to some epsilon value **e**. Using `max(x,e)` creates a discontinuity. So instead, let **m** be a blending width, and smoothly blend the epsilon **e** into the signal approaches as it approaches zero:

```
float almostIdentity( float x, float m, float
{
    if( x>m ) return x;
    float a = 2.0*e - m;
    float b = 2.0*m - 3.0*e;
    float t = x/m;
    return (a*t+b)*t*t + e;
}
```

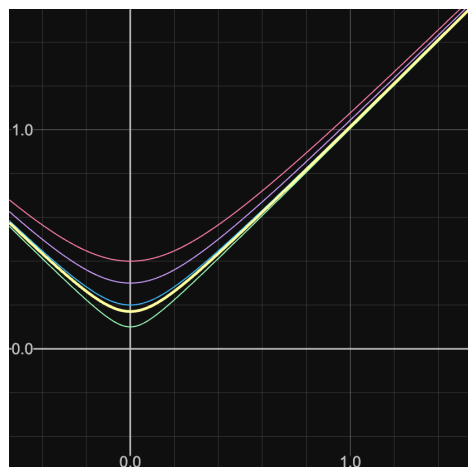


Smooth Identity 2/ Smooth abs()

[graphoy.com](https://www.graphoy.com) example

Another piece-wise function to get a smooth identity or smooth absolute value, is this one, which happens to be the one we use for the traditional smooth minimum of SDFs (the quadratic polynomial):

```
float smoothAbs( float x, float n )
{
    float x2 = x*x;
    float n2 = n*n;
    return x2<n2 ? (x2+n2)/(2.0*n) : abs(x);
}
```



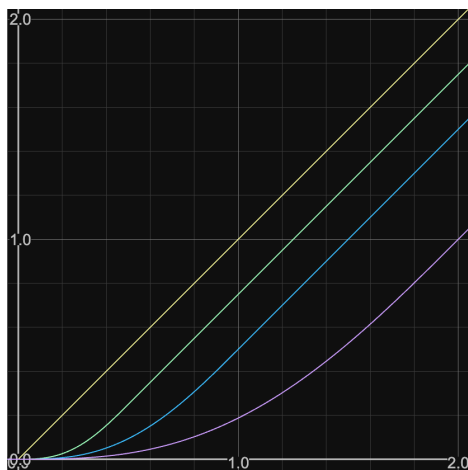
Smooth Identity / Smooth abs 2()

[Youtube explanation](#)

[graphoy.com](https://www.graphoy.com) example

A different way to achieve a near identity is through the square root of a biased square. I saw this technique first in a shader by user "omeometo" in Shadertoy. This approach can be a bit slower than the cubic above, depending on the hardware, but I find myself using it a lot these days, specially for "smooth mirroring" shapes, since it behaves almost like the absolute value of x. While it has zero derivative, it has a non-zero second derivative, so keep an eye in case it causes problems in your application.

```
float smoothAbs( float x, float n )
{
    return sqrt(x*x+n*n);
}
```



Smoothstep Integral

[Youtube explanation](#)

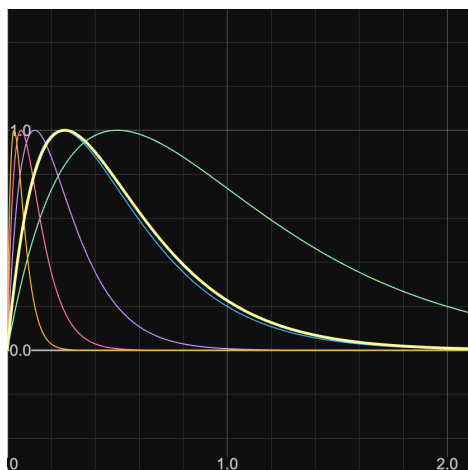
[graphtoy.com example](#)

If you use smoothstep for a velocity signal (say, you want to smoothly accelerate a stationary object into constant velocity motion), you need to integrate smoothstep() over time in order to get the actual position of value of the animation.

The function below is exactly that, the position of an object that accelerates with smoothstep. Note its derivative is never larger than 1, so no decelerations happen.

```
float integralSmoothstep( float x, float T )
{
    if( x>T ) return x - T/2.0;
    return x*x*x*(1.0-x*0.5/T)/T/T;
}
```

Impulses

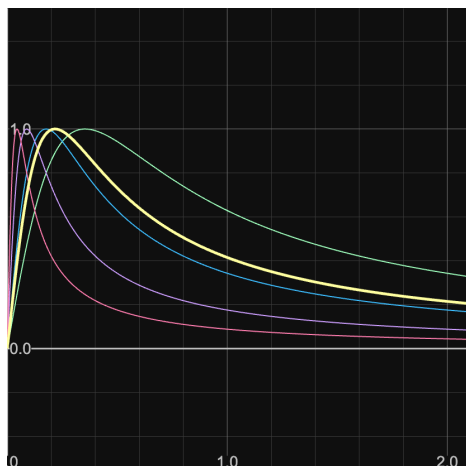


Exponential Impulse

[graphtoy.com example](#)

Impulses are great for triggering behaviours or making envelopes for music or animation. Basically, for anything that grows fast and then decays slowly. The following is an exponential impulse function. Use **k** to control the stretching of the function. Its maximum, which is 1, happens at exactly $x = 1/k$.

```
float expImpulse( float x, float k )
{
    float h = k*x;
    return h*exp(1.0-h);
}
```



Polynomial Impulse

graphtoy.com example

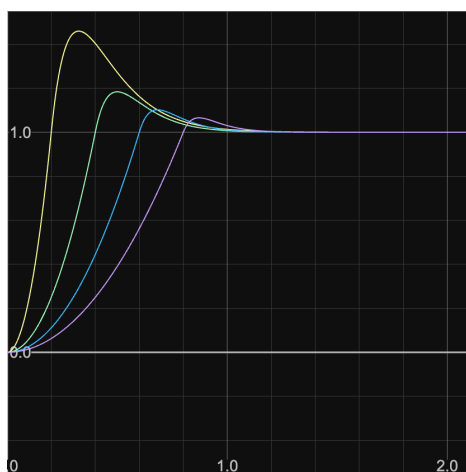
Another impulse function that doesn't use exponentials can be designed by using polynomials. Use k to control falloff of the function. For example, a quadratic can be used, which peaks at $x = \sqrt{1/k}$.

```
float quaImpulse( float k, float x )
{
    return 2.0*sqrt(k)*x/(1.0+k*x*x);
}
```

You can easily generalize it to other powers to get different falloff shapes, where n is the degree of the polynomial:

```
float polyImpulse( float k, float n, float x
{
    return (n/(n-1.0))*
        pow((n-1.0)*k, 1.0/n)*
        x/(1.0+k*pow(x,n));
}
```

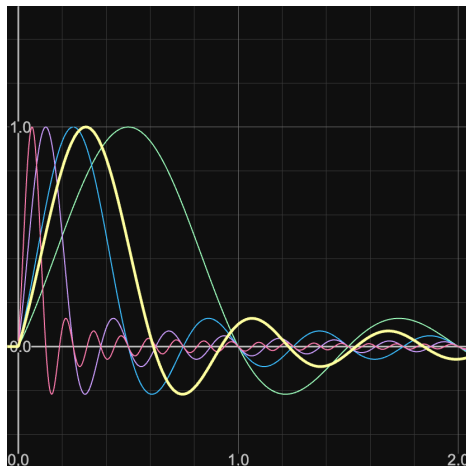
These generalized impulses peak at $x = [k(n-1)]^{-1/n}$.



Sustained Impulse

Similar to the previous, but it allows for control on the width of attack (through the parameter " k ") and the release (parameter " f ") independently. Also, the impulse releases at a value of 1 instead of 0.

```
float sustainedImpulse( float x, float f, flo
{
    float s = max(x-f,0.0);
    return min( x*x/(f*f), 1.0+(2.0/f)*s*exp(
}
```

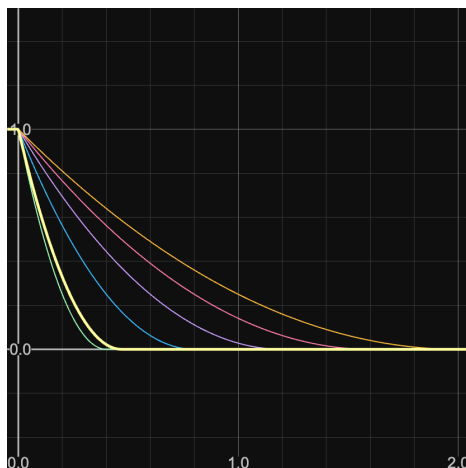


Sinc Impulse

[graphtoy.com example](#)

A phase shifted **sinc curve** can be useful if it starts at zero and ends at zero, for some bouncing behaviors (suggested by Hubert-Jan). Give **k** different integer values to tweak the amount of bounces. The functions max value is 1.0, but it can take negative values, which can make it unusable in some applications.

```
float sinc( float x, float k )
{
    float a = PI*(k*x-1.0);
    return sin(a)/a;
}
```



Falloff

[graphtoy.com example](#)

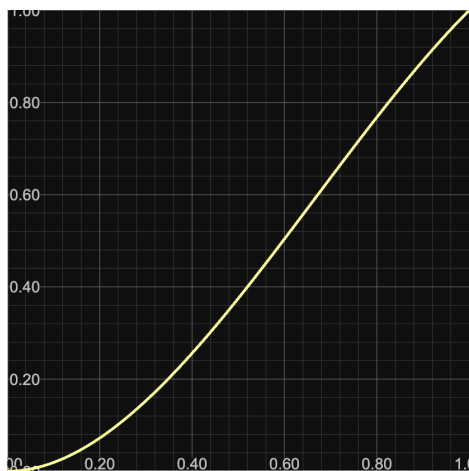
A quadratic falloff, like those in physically based point lights, but reaching zero at a given distance "m" rather than just asymptotically reaching it at infinity. Great for range controlled shadows, etc.

```
float trunc_fallof( float x, float m )
{
    if( x>m ) return 0.0;
    x /= m;
    return (x-2.0)*x+1.0;
}
```

Unitary remappings

These functions below remap the [0,1] interval into the [0,1] interval. They can be used to adjust image

contrasts, shape terrain slopes, modulate movements, sculpt forms, etc. One such a common function is the `smoothstep()`, which I don't include here for it is ubiquitous.

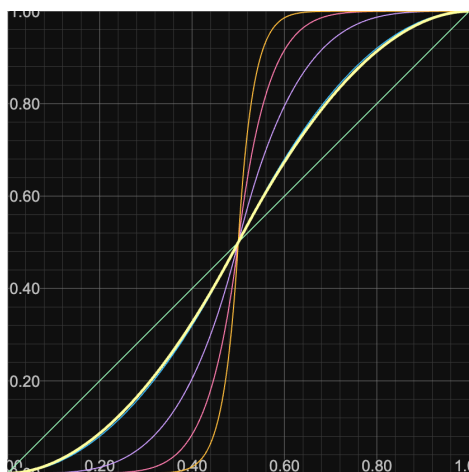


Almost Unit Identity

[graphtoy.com example](#)

This is a near-identity function that maps the unit interval into itself. It is the cousin of `smoothstep()`, in that it maps 0 to 0, 1 to 1, and has a 0 derivative at the origin, just like `smoothstep`. However, instead of having a 0 derivative at 1, it has a derivative of 1 at that point. It's equivalent to the Almost Identity above with $n=0$ and $m=1$. Since it's a cubic just like `smoothstep()` it is very fast to evaluate:

```
float almostUnitIdentity( float x )
{
    return x*x*(2.0-x);
}
```



Gain

[graphtoy.com example](#)

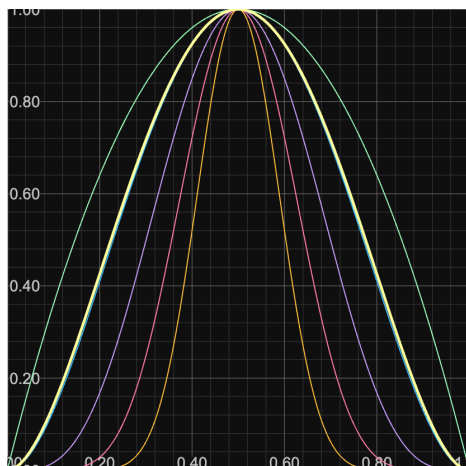
Remapping the unit interval into the unit interval by expanding the sides and compressing the center, and keeping 1/2 mapped to 1/2, that can be done with the `gain()` function. This was a common function in RSL tutorials (the Renderman Shading Language). $k=1$ is the identity curve, $k<1$ produces the classic `gain()` shape, and $k>1$ produces "s" shaped curves. The curves are symmetric (and inverse) for $k=a$ and $k=1/a$.

```
float gain( float x, float k )
{
    float a = 0.5*pow(2.0*((x<0.5)?x:1.0-x),
```

```

    return (x<0.5)?a:1.0-a;
}

```



Parabola

graphtoy.com example

A nice choice to remap the 0..1 interval into 0..1, such that the corners are mapped to 0 and the center to 1. You can then rise the parabola to a power k to control its shape.

```

float parabola( float x, float k )
{
    return pow( 4.0*x*(1.0-x), k );
}

```



Power Curve

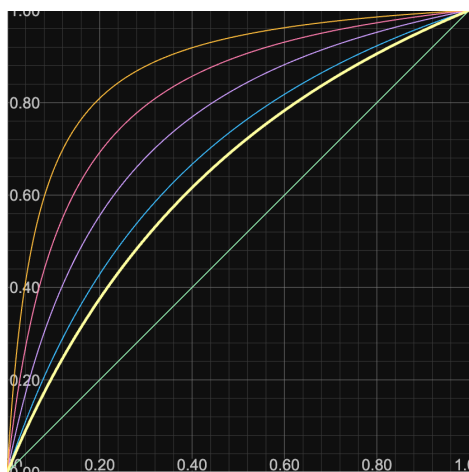
This is a generalization of the Parabola() above. It also maps the 0..1 interval into 0..1 by keeping the corners mapped to 0. But in this generalization you can control the shape one either side of the curve, which comes handy when creating leaves, eyes, and many other interesting shapes.

```

float pcurve( float x, float a, float b )
{
    float k = pow(a+b,a+b)/(pow(a,a)*pow(b,b));
    return k*pow(x,a)*pow(1.0-x,b);
}

```

Note that k is chosen such that the curve reaches exactly 1 at its maximum for illustration purposes, but in many applications the curve needs to be scaled anyways so the slow computation of k can be simply avoided.

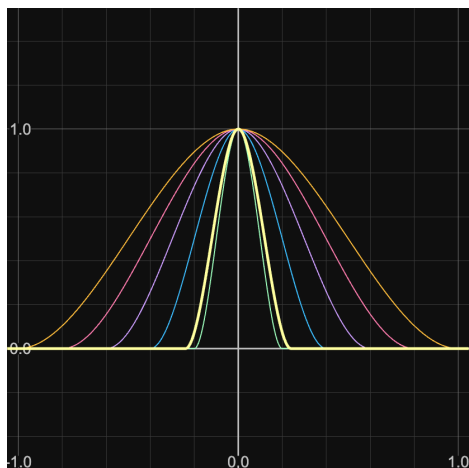


Tonemap

This function maps the 0 to 0 and 1 to 1, while rising the middle tones upwards, similar to a power function with exponent smaller than 1. However, when the numerator is not $k+1$ but something else (usually larger), this function is often used as a color tonemapping transfer function (similar to the Reinhard tonemapper), hence the name. Usually $k > 0$, but you can make it between 0 and -1 to bend the curves inwards like a positive power curve.

```
float tone( float x, float k )
{
    return (k+1.0)*x/(1.0+k*x);
}
```

Bumps

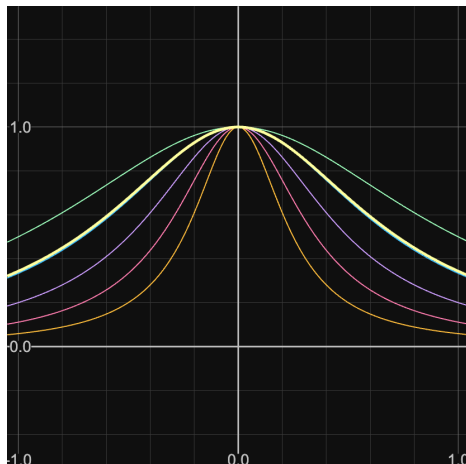


Cubic Pulse

graphtoy.com example

Chances are you found yourself doing $\text{smoothstep}(c-w, c, x)$ - $\text{smoothstep}(c, c+w, x)$ very often as a way to select a region centered at c that goes from $c-w$ to $c+w$. I know I do, and that's why I made this `cubicPulse()` below. You can also use it as a replacement for a gaussian with local support.

```
float cubicPulse( float x, float c, float w )
{
    x = abs(x - c);
    if( x > w ) return 0.0;
    x /= w;
    return 1.0 - x*x*(3.0-2.0*x);
}
```

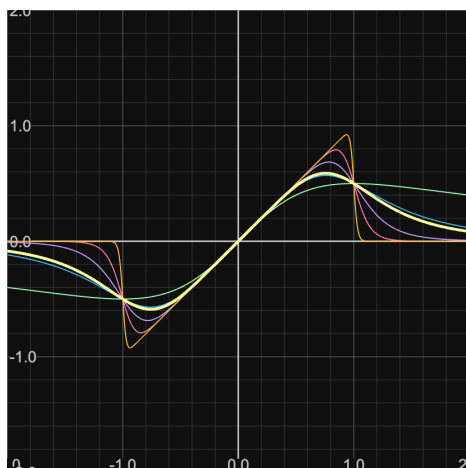


Rational Bump

[graphtoy.com example](#)

This function can also be a replacement for a gaussian sometimes if you can do with infinite support, ie, this function never reaches exactly zero no matter how far you go in the real axis:

```
float rationalBump( float x, float k )
{
    return 1.0/(1.0+k*x*x);
}
```



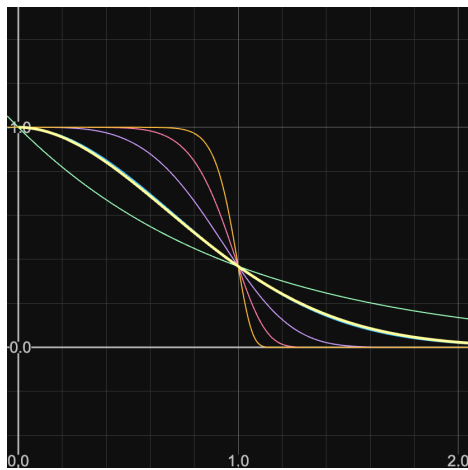
Double Bump

[graphtoy.com example](#)

This is a double bump with controllable attenuation. The way it's listed below, the derivative at 0 is always 1, which makes it perfect to create smooth RELU functions if you attach a $y=x$ segment when x is positive. If on the other hand you don't care about the inclination at the origin and instead you want to normalize it so it always peaks with a value of 1.0, then you can multiply the function above by $k/\text{pow}((k-1),(1-1/k))$.

```
float doubleBump( float x, float k )
{
    return x/(1.0+pow(x,k)) );
}
```

Sigmoids / Steps



Exponential Step

[graphtoy.com example](#)

A natural attenuation is an exponential of a linearly decaying quantity. A gaussian, is an exponential of a quadratically decaying quantity: purple curve, $\exp(-x^2)$. You can generalize and keep increasing powers, and get a sharper and sharper s-shaped curves. For really high values of n you can approximate a perfect step().

```
float expStep( float x, float k )  
{  
    return exp2(-pow(x,k));  
}
```

[inigo quilez](#) - learning computer graphics since 1994